

**Lembram do Jogo do Bicho Misterioso? E se o computador escolhesse as perguntas sozinho?**



# O que é uma árvore de decisão

É uma sequência de perguntas de sim/não que vai eliminando candidatos até sobrar uma resposta — exatamente a lógica que vocês escreveram na mão na Aula 01.

A diferença: agora o algoritmo escolhe sozinho quais perguntas fazer, e em que ordem, olhando pros dados.

**Nó raiz (primeira pergunta) → ramos (respostas) → folhas (decisão final).**



# Vamos ver na prática

```
import json
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

animais = json.load(open("animais_completo.json", encoding="utf-8"))
df = pd.DataFrame(animais)

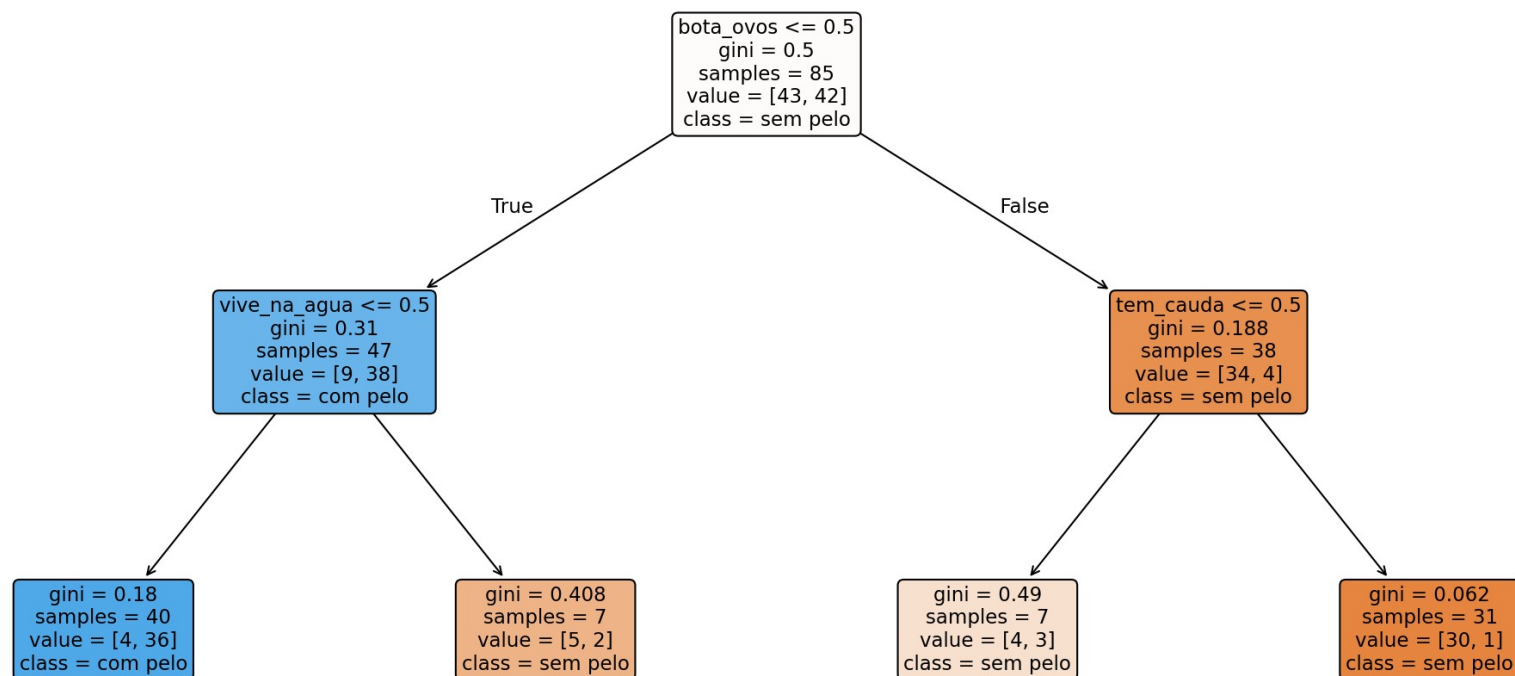
features = ["voa", "nada", "bota_ovos", "tem_quatro_patas",
            "vive_na_agua", "e_carnivoro", "tem_cauda", "e_domestico"]
X = df[features]
y = df["tem_pelos"]

modelo = DecisionTreeClassifier(max_depth=2, random_state=42)
modelo.fit(X, y)
```



# A árvore que o algoritmo aprendeu

Árvore treinada nos 85 animais (prevendo tem\_pelos)



# Vamos ver na prática

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(14,7))
plot_tree(modelo, feature_names=features,
          class_names=["sem pelo", "com pelo"],
          filled=True, rounded=True)
plt.show()
```



# Os erros da árvore

Ela erra em 10 dos 85 animais — incluindo casos genuinamente ambíguos: porco (pelo esperso), aranha e abelha ('peludas' mas não mamíferos), foca e leão-marinho (pelo, mas totalmente aquáticos).

O caso mais interessante: o ornitorrinco. É mamífero, tem pelo — mas bota ovos, e a árvore aprendeu que 'bota\_ovos' é o sinal mais forte de 'sem pelo'. Um mamífero que quebra essa regra confunde o modelo.

**Nenhum modelo é perfeito — e os erros costumam acontecer exatamente nos casos mais raros e ambíguos da natureza.**

# Como o algoritmo decide onde perguntar

A cada divisão possível, o algoritmo mede a impureza de Gini — o quão ‘misturadas’ as classes ficam. Fórmula:  $Gini = 1 - \sum(p_i^2)$ , onde  $p_i$  é a proporção de cada classe no grupo.

Gini não é sigla — é sobrenome de *Corrado Gini*, estatístico italiano (1884-1965) que criou uma fórmula parecida pra medir desigualdade de renda entre países — depois adaptada pra medir ‘mistura de classes’ em árvores.

O algoritmo escolhe a pergunta que mais reduz essa impureza.

**Mesmo papel que o erro quadrático teve na regressão, e o log loss na regressão logística.**

# Gini, com números reais

Nó raiz da nossa árvore: 43 animais sem pelo, 42 com pelo (quase 50/50) →  $\text{Gini} = 1 - (0,506^2 + 0,494^2) = 0,500$  — bem impuro.

Uma das folhas: 30 sem pelo, 1 com pelo (bem homogênea) →  $\text{Gini} = 1 - (0,968^2 + 0,032^2) = 0,062$  — quase puro.

Repare: esses são exatamente os números que já apareciam desenhados na árvore que vimos antes.

**Quanto mais perto de 0, mais 'decidida' a folha — quanto mais perto de 0,5, mais incerta.**





# O papel da profundidade

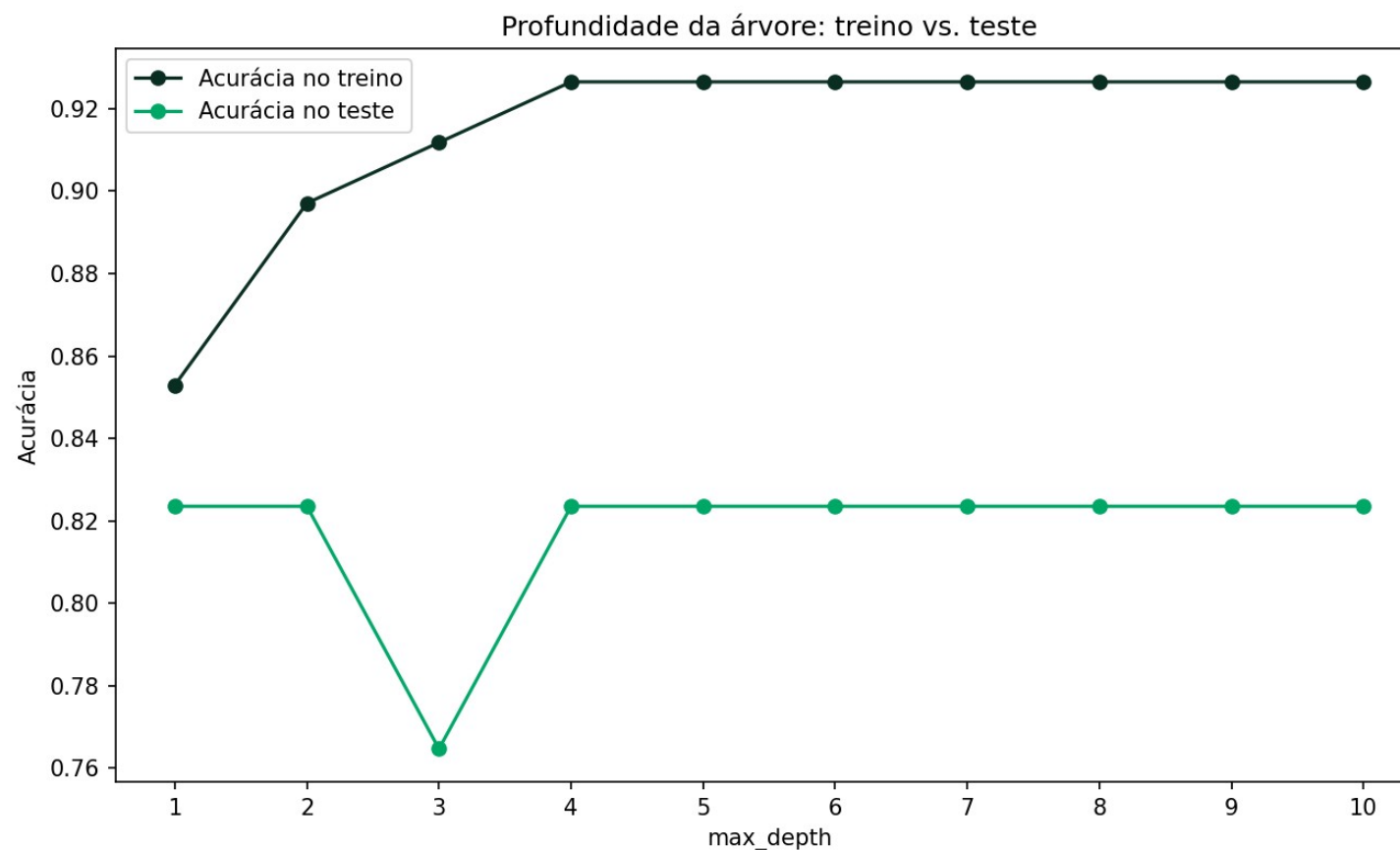
Muito rasa (poucas perguntas): erra por simplicidade demais, não captura padrões reais.

Muito funda: decora o treino, pergunta específica demais pra generalizar bem.

**Mesmo equilíbrio que já vimos com grau do polinômio e K do KNN.**



# Profundidade: treino vs. teste



# Vamos ver na prática

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=5, stratify=y
)

profundidades = range(1, 11)
acc_treino, acc_teste = [], []
for d in profundidades:
    m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
    acc_treino.append(accuracy_score(y_train, m.predict(X_train)))
    acc_teste.append(accuracy_score(y_test, m.predict(X_test)))

plt.plot(list(profundidades), acc_treino, marker="o", label="Treino")
plt.plot(list(profundidades), acc_teste, marker="o", label="Teste")
plt.legend()
plt.show()
```



# O fluxo no Scikit-Learn

Mesma gramática de sempre: `DecisionTreeClassifier(max_depth=D)`, `fit()`, `predict()` — já bem familiar a essa altura.

**A única novidade real é o `max_depth` como hiperparâmetro a ajustar.**



# Vamos ver na prática

```
modelo_final = DecisionTreeClassifier(max_depth=3, random_state=42)
modelo_final.fit(X_train, y_train)

pred = modelo_final.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, pred):.2%}")
```



# Importância das features

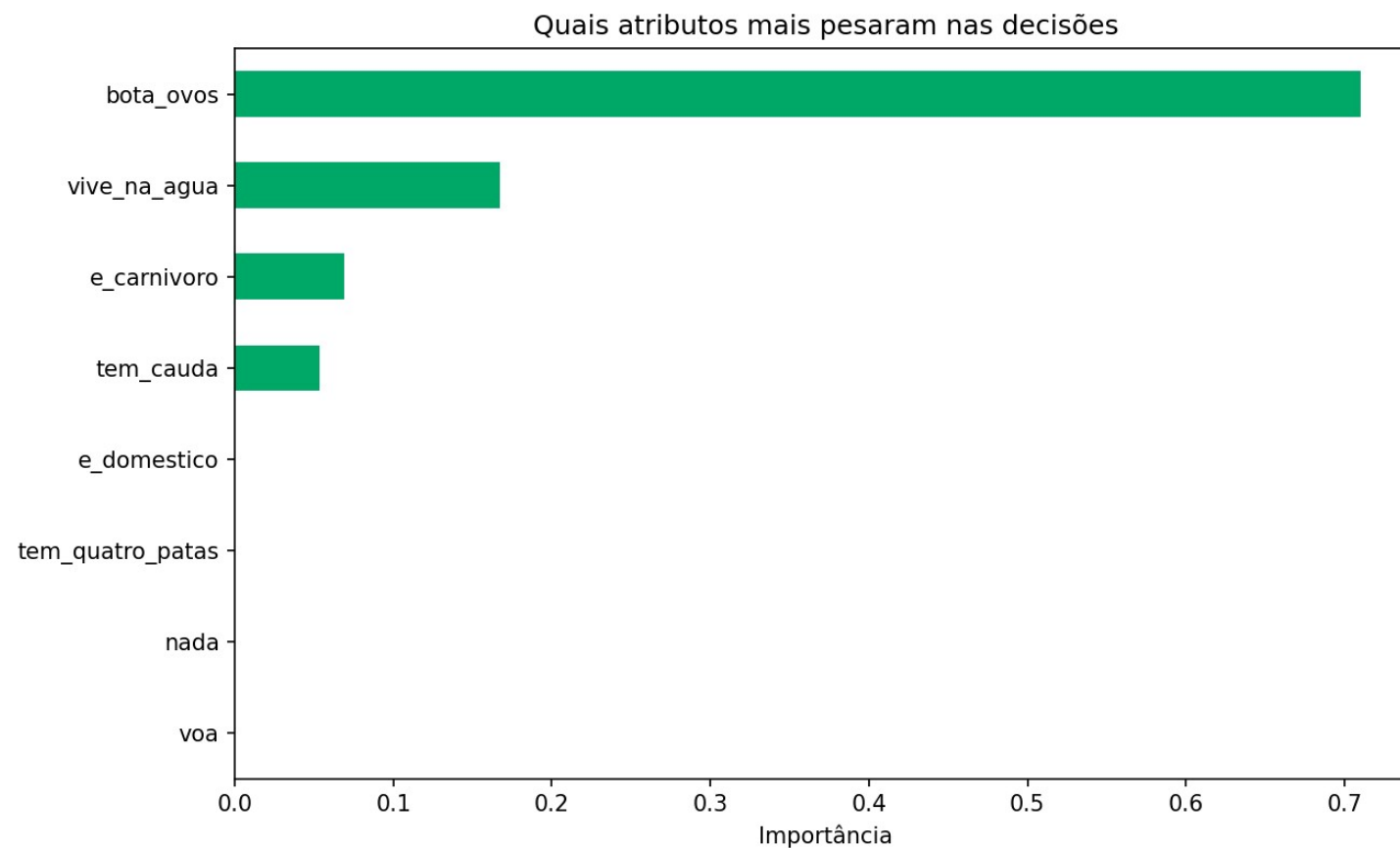
`feature_importances_` mostra quais variáveis mais pesaram nas decisões da árvore inteira — não só na primeira pergunta.

É um bônus que só árvore (e Random Forest) têm — nem regressão logística nem KNN oferecem isso diretamente.

**Ótimo pra explicar o modelo pra outras pessoas, não só pra prever.**



# Quais atributos mais pesaram



# Vamos ver na prática

```
importancias = pd.Series(  
    modelo_final.feature_importances_, index=features  
)  
.sort_values()  
  
importancias.plot(kind="barh")  
plt.show()
```





## FECHANDO ÁRVORE DE DECISÃO

# Perguntas aprendidas, não programadas

- 1 **Aprendizado automático de perguntas** — Mesma lógica do Jogo do Bicho, agora sem código escrito na mão
- 2 **Gini escolhe onde perguntar** — A pergunta que mais separa bem as classes
- 3 **Profundidade é o equilíbrio** — Raso demais erra; fundo demais decora

E se, em vez de 1 árvore, usássemos várias e deixássemos elas votarem? Random Forest.

